ATIVIDADE — ARQUITETURA DE SISTEMAS WEB

Padrão MVC aplicado ao sistema “Biblioteca Virtual” — turma de Programação Web

ATIVIDADE — ARQUITETURA DE SISTEMAS WEB

Padrão MVC aplicado ao sistema "Biblioteca Virtual"

> **Disciplina:** Programação Web / Sistemas Web | **Turma:** _____ | **Valor:** ____
> **Prazo de entrega:** _____ | **Modo**: individual ou dupla (conforme combinado em aula)

---

1. Contexto (Cenário Único)

Você foi contratado(a) como **arquiteto(a) de software de uma consultoria**. A escola precisa de um laudo técnico sobre a arquitetura do sistema que a turma desenvolveu ao longo do semestre: a **Biblioteca Virtual**.

O sistema atual é um site web para gerenciamento de um acervo de livros, com **listagem do catálogo** e **cadastro de novos títulos** (título, autor, sinopse e capa). Ele foi construído com **Node.js, Express, EJS, MySQL (via Knex) e Bootstrap**.

Contexto e restrições:

- Servidor local/simples; poucos usuários simultâneos (secretaria e visitantes), acervo na casa das centenas de títulos.
- Acesso **somente pelo navegador** (web responsiva) — sem aplicativo nativo.
- Equipe pequena (2 desenvolvedores) e prazo curto; stack deve ser **gratuita e open source**.
- Os dados hoje são públicos (catálogo), mas em breve serão necessários **controle de acesso por perfis** (aluno, funcionário, administrador) para empréstimos e gestão.
- **Atenção à LGPD**: futuramente haverá dados pessoais (usuários, empréstimos).

> **Importante:** esta é uma atividade de **análise e documentação**, não de programação. Leia o código-fonte existente, mas **não é necessário (nem permitido) alterá-lo** para responder.

---

2. Objetivo pedagógico — Foco na pesquisa sobre o padrão MVC

O coração desta atividade é a **pesquisa sobre o padrão arquitetural MVC (Model-View-Controller)**. Você deve:

1. Estudar o padrão a fundo (conceito, origem, camadas, prós, contras e quando usar);
2. **Aplicá-lo ao projeto real** da Biblioteca Virtual, identificando onde cada camada vive no código;
3. Discutir as decisões técnicas do projeto e as alternativas descartadas — sempre justificando o **porquê**;
4. Comparar o modelo **Tradicional** (renderização no servidor) com o modelo **REST**, posicionando o projeto nessa comparação.

---

3. Entregáveis

Entregável 1 — Pesquisa sobre o padrão MVC

Redija um texto (com parágrafos próprios, **não copy-paste**) cobrindo:

- O que é o padrão MVC e qual problema ele resolve;
- **Origem/história** e quem criou (pesquise: Trygve Reenskaug, Smalltalk-80, década de 1970/80);
- O papel de **cada camada**: *Model*, *View* e *Controller*;
- O **fluxo de uma requisição** no MVC (navegador → rota → controlador → modelo → banco → visão → navegador);
- **Vantagens** e **desvantagens** do MVC;
- **Quando usar** e **quando não usar** esse padrão;
- Referências utilizadas (**mínimo 3 fontes confiáveis** — MDN, Refactoring Guru, Martin Fowler, documentação do Express, sites acadêmicos, etc.).

Entregável 2 — Mapeamento do projeto no MVC

Preencha um quadro indicando, para cada camada, **qual arquivo/pasta do projeto Biblioteca Virtual** a representa e **qual a sua função**, com justificativa:

Camada	Arquivo(s) / pasta no projeto	Função	Justificativa
View (Visão)	`src/views/...`	...	...
Controller (Controle)	`src/controllers/...`	...	...
Model (Modelo)	`src/models/...`	...	...
Rotas (dispatcher)	`src/routes/...`	...	...
Conexão/Base	`src/config/...`	...	...
Ponto de entrada	`index.js`	...	...

Depois, faça uma **análise crítica** respondendo:

- O projeto **segue bem** o MVC? *Por quê?*
- Onde ele **fura** o padrão? (ex.: lógica "pesada" concentrada no controller, ausência de validação, URLs de capa fixas no código)
- O que você faria de diferente para melhorar essa arquitetura?

Entregável 3 — Diagrama geral da arquitetura

Crie um **diagrama de camadas + fluxo**: do navegador até o banco de dados, mostrando **onde cada arquivo do projeto entra** no caminho (ex.: `web.js` → `BookController.js` → `Book.js` /Knex → MySQL → `home.ejs`).

Requisitos do diagrama:

- Deve estar **legível** (sem cortes nem borrões no PDF);
- Pode ser desenhado ou feito como código (diagrams.net, Excalidraw, Mermaid, PlantUML);
- Identifique claramente: as camadas, a direção do fluxo e os arquivos envolvidos.

Entregável 4 — Quadro de decisões técnicas

Preencha o quadro abaixo. Em **toda linha**, explique o motivo da escolha **e por que a alternativa foi descartada**.

Item	Escolha	Motivo	Alternativa descartada e por quê
Padrão arquitetural	MVC (camadas Model-View-Controller)	...	Arquitetura "tudo numa página" (sem separação) ou em camadas sem MVC, e por quê
Modelo de renderização	Tradicional (EJS no servidor)	...	SPA (React/Vue + API REST), e por quê
Linguagem (back-end)	Node.js	...	PHP (Laravel), Python (Django/Flask), Java (Spring), e por quê
Framework (back-end)	Express	...	Fastify, NestJS, ou outro, e por quê
Front-end	Bootstrap + HTML/EJS (server-side)	...	Tailwind, CSS puro, frameworks JS, e por quê
Banco de dados	MySQL	...	PostgreSQL, SQLite, MongoDB (NoSQL), e por quê
Acesso a dados	Knex (query builder)	...	ORM completo (Sequelize/Prisma) ou SQL puro, e por quê
Autenticação (futuro)	*sua proposta*	...	*alternativa(s) descartada(s)*
Infraestrutura/deploy	*sua proposta* (ex.: local + hospedagem gratuita)	...	*alternativa(s) descartada(s)*

> *Dica:* nos itens já implementados, baseie-se no código real; nos itens futuros (autenticação, deploy), proponha a melhor solução e justifique.

Entregável 5 — Justificativas e trade-offs

Para **cada decisão do quadro** (Entregável 4), escreva em prosa:

1. **Vantagens** da escolha;
2. **Desvantagens** ou limitações;
3. **O que foi descartado** e em que situação valeria a pena trocar de solução.

Entregável 6 — Matriz comparativa Tradicional × REST

Complete a matriz abaixo e depois escreva a conclusão:

Critério	Modelo Tradicional (renderização no servidor)	Modelo REST (API + cliente)	O que o projeto usa
Onde o HTML é gerado			
Comunicação cliente-servidor			
Separação front-end × back-end			
SEO / indexação			
Manutenção e complexidade			
Consumo por apps nativos			
Tempo até a primeira tela			

Conclusão obrigatória (mín. 1 parágrafo): o projeto adota qual modelo? Por quê? Em que **evolução futura** (ex.: app móvel, multi-cliente) faria sentido migrar para o outro?

Entregável 7 — Conclusão e referências

- **Conclusão geral:** o MVC foi uma boa escolha para a Biblioteca Virtual? O que você aprenderia com este laudo?
- **Referências** completas, caprichadas, das fontes consultadas (formato a ser combinado em aula).

---

4. Formato de entrega (escolher UMA opção)

Opção A — Relatório em PDF

- **Capa:** nome(s), turma, disciplina, data e título;
- **Sumário;**
- **Entregáveis 1 a 7;**
- **Conclusão** e **referências**;
- Diagramas **legíveis** (sem cortes ou borrões).

Opção B — Apresentação em vídeo

- **Duração:** 5 a 8 minutos;
- **Roteiro sugerido:**

1. Contexto do sistema Biblioteca Virtual — 1 min;
2. Pesquisa sobre MVC + mapeamento do projeto — 2 a 3 min;
3. Diagrama, decisões técnicas, trade-offs e matriz Tradicional × REST — 2 a 3 min;
4. Conclusão/trade-offs — 1 min.

- Pode ser gravação de tela com slides, ou diagramas desenhados e explicados;
- Enviar como **link** (YouTube não listado, Google Drive ou OneDrive).

---

5. Critérios de avaliação

Critério	Peso
Pesquisa sobre o MVC: profundidade, corretude e uso de referências	2,5
Mapeamento do projeto no MVC + análise crítica	2,0
Diagrama geral da arquitetura (legibilidade, camadas e fluxo)	1,5
Quadro de decisões e trade-offs (coerência e justificativas reais)	2,0
Matriz Tradicional × REST e conclusão	1,0
Formatação do documento (capa, sumário, organização, referências)	1,0
Total	**10,0**

---

6. Ferramentas sugeridas

- **Diagramas:** diagrams.net (draw.io), Excalidraw, Lucidchart, Whimsical, Figma, Canva;
- **Diagramas como código:** Mermaid, PlantUML;
- **Documento/PDF:** Google Docs, Word, Notion, Markdown + Pandoc, Canva;
- **Vídeo (opção B):** OBS Studio, Loom, Clipchamp, CapCut, DaVinci Resolve;
- **Colaboração/versão:** GitHub, GitLab, Google Drive.

---

7. Referências iniciais para a pesquisa

- MDN Web Docs — *Understanding Client-Side JS Frameworks / MVC* – https://developer.mozilla.org
- Refactoring Guru — *Model-View-Controller* – https://refactoring.guru
- Martin Fowler — *Model-View-Controller* – https://martinfowler.com
- Express (documentação oficial) – https://expressjs.com
- Knex.js (documentação oficial) – https://knexjs.org
- Roadmap.sh — backend / arquitetura – https://roadmap.sh

Busque também artigos acadêmicos e o clássico artigo original de Trygve Reenskaug sobre o MVC.
